Root Root Root Root

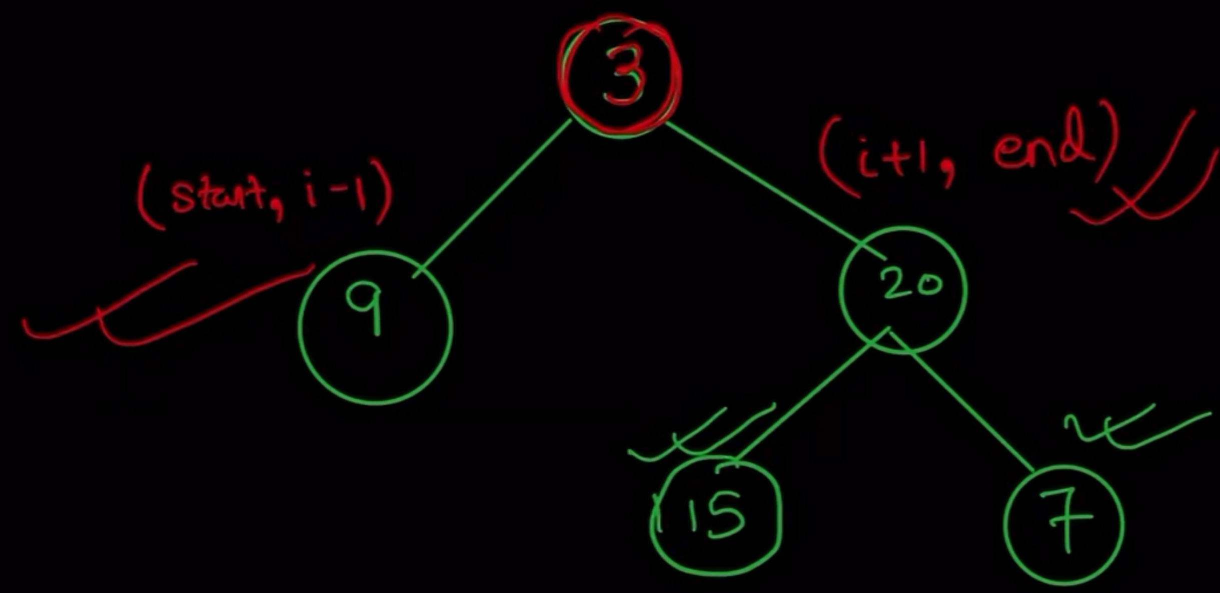
Preorder = {3, 9, 20, 15, 7}

Root, left, Right

start i end

Inorder = {9, $\boxed{3}$, 15, 20, 7}

Left, Root, Right



idx

	0	1	2	3	4
Pre:	3	9	20	15	7

start, end

	0	1	2	3	4
In:	9	3	15	20	7

i

root = 3

root->left = func(start, i-1);

root->right = func(i+1, end;

return root;

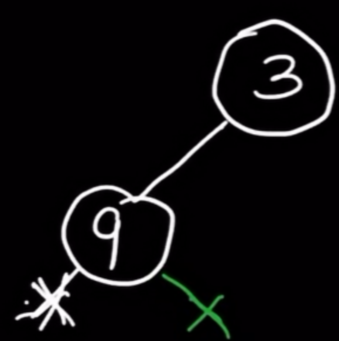
Pre:

0	1	2	3	4
3	9	20	15	7

In:

start 0	1	2	3	4 end
9	3	15	20	7

i



① idx = 0
root = 3
i = 1
idx++;
root→left = (start, i-1) — (0, 0)
root→right = (i+1, end) — (2, 4)

② idx = 1
start = 0
end = 0
root = 9
i = 0
idx++;
root→left = (0, −1) X
root→right = (1, 0) X

i = 0

idx++;

root->left = (0, -1) X

root->right = (1, 0) X

3 idx = 2

start = 2
end = 4

root = 20

i = 3

idx++;

root->left = (2, 2) 15

root->right = (4, 4) 7

Construct Binary Tree From Preorder And Inorder Traversal | Leetcode 105 | codestorywithMIK

2.10

leetcode.com

Construct Binary Tree from Preorder and Inorder Traversal - LeetCode

LeetCode Explore Problems Interview New Contest Discuss Store Premium 0

C++ Autocomplete

```cpp
*/
class Solution {
public:

    TreeNode* solve(vector<int>& preorder, vector<int>& inorder, int start, int end, int& idx) {
        if(start > end)
            return NULL;

        int rootVal = preorder[idx];
        int i = start;

        for(; i <= end; i++) {
            if(inorder[i] == rootVal)
                break;
        }

        idx++;
        TreeNode* root = new TreeNode(rootVal);
        root->left  = solve(preorder, inorder, start, i-1, idx);
        root->right = solve(preorder, inorder, i+1, end, idx);

        return root;
    }

    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        int n = preorder.size();

        int idx = 0;

        return solve(preorder, inorder, 0, n-1, idx);
    }
};
```

Run Example Testcases

Console Contribute

23:15 / 24:26

Run Code Submit